

Archivos de Audio MP3: Codificación y su Implementación en Microcontroladores

Contreras R Orlando^{1,2*}

³Department of Electronic Systems, UAA, Av. Universidad 940,
Aguascalientes, 20131, Aguascalientes, México.

Corresponding author(s). E-mail(s): [al348390,al464376}@edu.uaa.mx](mailto:{al348390,al464376}@edu.uaa.mx);

Abstract

El presente reporte estudia el formato de audio comprimido *MPEG-1/2 Audio Layer III* (MP3), desarrollado principalmente por el Fraunhofer Institut für Integrierte Schaltungen (IIS) bajo el liderazgo de Karlheinz Brandenburg. Se abordan los fundamentos psicoacústicos que sustentan la compresión con pérdida, la estructura binaria de la trama MP3, la arquitectura interna del codificador (banco de filtros híbrido, MDCT, modelo psicoacústico, cuantización no uniforme, codificación de Huffman y reservorio de bits) y la del decodificador correspondiente. Se presentan además dos estrategias de implementación práctica en microcontroladores: decodificación por hardware dedicado mediante el códec VS1053b vía SPI, y decodificación por software utilizando el decodificador de punto fijo Helix sobre un periférico I2S con salida DMA. Finalmente se discuten ventajas y desventajas frente a otros formatos de audio, sus aplicaciones actuales, y se listan recursos verificados para su implementación en plataformas STM32/HAL.

Keywords: MP3, MPEG-1 Layer III, Codificación de audio, Psicoacústica, MDCT, Huffman, VS1053, STM32, I2S, Helix.

1 Introducción

El formato **MP3** (*MPEG-1 Audio Layer III*, y posteriormente *MPEG-2 Audio Layer III*) es un esquema de compresión de audio con pérdida diseñado para reducir drásticamente el tamaño de los archivos de audio digital manteniendo una calidad percibida cercana a la del material sin comprimir. A diferencia de la compresión sin pérdida (como FLAC), MP3 aprovecha las limitaciones del sistema auditivo humano – el llamado **modelo psicoacústico** – para descartar de forma deliberada e irreversible la información que un oyente promedio no percibiría, logrando reducciones de tamaño de entre el 75% y el 95% respecto al audio de calidad CD, dependiendo de la tasa de bits elegida.

Puntos clave que se desarrollan en este reporte:

- MP3 combina un **banco de filtros híbrido** (polifase + MDCT) con un **modelo psicoacústico** para concentrar los bits disponibles en las regiones del espectro donde el oído es más sensible.
- La trama MP3 es un contenedor autocontenido de 32 bits de cabecera más datos de audio, lo que permite decodificar el flujo a partir de cualquier punto (streaming).
- En microcontroladores, la decodificación de MP3 puede resolverse mediante un **chip decodificador dedicado** (descarga completa de cómputo) o mediante un **decodificador por software de punto fijo**, ejecutado directamente en el núcleo Cortex-M.
- Todas las patentes asociadas a MP3 expiraron en 2017, por lo que su codificación y decodificación son hoy de uso libre.

MP3 sigue siendo relevante en el ámbito de sistemas embebidos porque, a diferencia de formatos más modernos como AAC u Opus, cuenta con decodificadores de referencia extremadamente maduros, ligeros y de código abierto, además de un ecosistema de silicio dedicado (como el VS1053b) pensado específicamente para microcontroladores de bajos recursos.

2 Breve Historia

El desarrollo del algoritmo que se convertiría en MP3 comenzó en 1987 como una colaboración entre el Fraunhofer IIS-A y la Universidad de Erlangen-Núremberg, dentro del marco del programa europeo EUREKA 147 orientado a la radiodifusión digital de audio (DAB). La propuesta de Fraunhofer, conocida como **ASPEC** (*Adaptive Spectral Perceptual Entropy Coding*), compitió contra otras arquitecturas –principalmente **MUSICAM**, de Philips y el Institut für Rundfunktechnik– dentro del proceso de estandarización que el *Moving Picture Experts Group* (MPEG) inició en 1988 bajo la ISO.

Tras evaluaciones comparativas, el MPEG decidió en 1990 combinar ambas propuestas en una familia de tres capas de codificación de complejidad creciente: la **Capa I** (una variante simplificada de MUSICAM), la **Capa II** (MUSICAM optimizado, adoptado por DAB) y la **Capa III** (basada en ASPEC, con mayor eficiencia de compresión a costa de mayor complejidad computacional). El estándar se formalizó en agosto de 1993 como **ISO/IEC 11172-3**, dentro del conjunto MPEG-1. En 1995 se publicó la extensión **ISO/IEC 13818-3** (MPEG-2), que añadió tasas de muestreo más bajas y compatibilidad retroactiva.

El nombre de archivo **.mp3** se adoptó formalmente el 14 de julio de 1995, mediante una votación interna en Fraunhofer IIS. La verdadera explosión de popularidad del formato llegó a partir de 1995 con la proliferación de Internet como medio de distribución de música, y se consolidó con la aparición de codificadores de alta calidad y libres de restricciones prácticas como **LAME** (1998), y de reproductores portátiles dedicados a finales de esa década. Es relevante destacar, desde una perspectiva de propiedad intelectual, que la totalidad de las patentes que protegían las técnicas de codificación y decodificación MP3 expiraron a nivel internacional el 23 de abril de 2017, lo que eliminó cualquier restricción de licenciamiento para su uso comercial o educativo.

Fig. 1: Línea de tiempo del desarrollo histórico del formato MP3

3 Conceptos Básicos

3.1 Compresión con Pérdida y Percepción Auditiva

MP3 es un codificador **perceptual**: no busca reconstruir la señal original de forma matemáticamente exacta, sino producir una señal que, al ser escuchada por un humano, resulte indistinguible (o casi) de la original. Esto se logra explotando dos fenómenos del sistema auditivo:

- **Enmascaramiento en frecuencia:** un tono intenso en una banda de frecuencia reduce la capacidad del oído para percibir tonos más débiles en bandas cercanas. El codificador calcula, para cada subbanda, un *umbral de enmascaramiento* y permite que el ruido de cuantización quede oculto bajo ese umbral.
- **Enmascaramiento temporal:** un sonido intenso enmascara sonidos más débiles que ocurren inmediatamente antes (*pre-enmascaramiento*) o después (*post-enmascaramiento*) de él en el tiempo, lo que permite relajar la precisión de codificación en esas ventanas temporales.

El **modelo psicoacústico** del codificador (el estándar define dos variantes, Modelo 1 y Modelo 2, siendo este último el más sofisticado) analiza el espectro de la señal y produce, por cada banda o grupo de bandas, una *relación señal-a-máscara* (SMR, *Signal-to-Mask Ratio*) que el bloque de asignación de bits utiliza para decidir cuántos bits invertir en cada región espectral.

3.2 Tasa de Bits: CBR, VBR y ABR

El estándar Layer III admite tasas de bits desde 8 hasta 320 kbit/s por canal, y permite que la tasa cambie de una trama a otra (*bitrate switching*). Esto da lugar a tres modos de operación comunes en los codificadores:

- **CBR (*Constant Bitrate*):** todas las tramas usan la misma tasa de bits nominal; simplifica el *seek* y el cálculo de duración, a costa de desperdiciar bits en pasajes simples y limitarlos en pasajes complejos.
- **VBR (*Variable Bitrate*):** el codificador asigna más bits a los segmentos perceptualmente complejos y menos a los silencios o segmentos simples, optimizando la relación calidad/tamaño total del archivo.
- **ABR (*Average Bitrate*):** un híbrido que intenta alcanzar una tasa promedio objetivo permitiendo variaciones locales, similar en espíritu a VBR pero con un tamaño de archivo más predecible.

3.3 Frecuencias de Muestreo Soportadas

Dependiendo de la versión del estándar, MP3 admite las siguientes frecuencias de muestreo: 32, 44.1 y 48 kHz bajo MPEG-1; 16, 22.05 y 24 kHz bajo MPEG-2; y 8, 11.025 y 12 kHz bajo la extensión no oficial conocida como “MPEG 2.5”, creada por Fraunhofer para cubrir aplicaciones de muy baja tasa de bits (por ejemplo, voz).

4 Estructura y Arquitectura Técnica

4.1 Estructura de la Trama (Frame)

Un archivo MP3 es, en esencia, una secuencia continua de **tramas** independientes entre sí; cada trama contiene un número fijo de muestras de audio (1152 para Layer III en MPEG-1) y puede, en principio, decodificarse sin necesidad de tramas anteriores

–propiedad clave que hace a MP3 idóneo para *streaming*–. Cada trama comienza con una **cabecera** de 32 bits, cuyos campos se describen en la Tabla 1.

Fig. 2: Estructura de bits de la cabecera de trama MP3 (32 bits)

Table 1: Campos de la cabecera de trama MP3 (ISO/IEC 11172-3)

Campo	Tamaño	Descripción
Syncword	11–12 bits	Patrón fijo de bits en 1 (0xFFE u 0xFFF) que marca el inicio de una trama válida.
Version ID	2 bits	Distingue MPEG-1, MPEG-2 y la extensión MPEG 2.5.
Layer	2 bits	Indica la capa de codificación (I, II o III).
Protection bit	1 bit	Si vale 0, la cabecera es seguida de un CRC de 16 bits; si vale 1, no hay CRC.
Bitrate index	4 bits	Índice a una tabla de tasas de bits válidas (8 a 320 kbit/s para Layer III).
Sampling rate index	2 bits	Índice a la frecuencia de muestreo (p. ej. 44.1 kHz).
Padding bit	1 bit	Indica si la trama incluye un byte de relleno adicional para cuadrar la duración exacta.
Private bit	1 bit	Sin uso definido por el estándar; libre para el codificador.
Channel mode	2 bits	Estéreo, estéreo conjunto (<i>joint stereo</i>), dual mono o mono.
Mode extension	2 bits	Parámetros adicionales de estéreo conjunto (<i>intensity stereo/M/S stereo</i>).
Copyright / Original	1 + 1 bits	Banderas informativas sin efecto en la decodificación.
Emphasis	2 bits	Curva de preénfasis aplicada (raramente usada en la práctica).

El tamaño en bytes de una trama Layer III (MPEG-1) se calcula como:

$$\text{FrameLen} = \left\lfloor \frac{144 \times \text{BitRate}}{\text{SampleRate}} \right\rfloor + \text{Padding} \quad (1)$$

Por ejemplo, para 192 kbit/s y 44.1 kHz con padding activo, $\text{FrameLen} = \lfloor 144 \times 192000 / 44100 \rfloor + 1 = 627$ bytes. Para MPEG-2/2.5 Layer III, donde cada trama contiene 576 muestras en lugar de 1152, la constante 144 se sustituye por 72.

A continuación de la cabecera (y del CRC opcional) se encuentra la **información lateral** (*side information*: 17 bytes en mono, 32 en estéreo), que incluye el puntero **main_data_begin** –un desplazamiento negativo en bytes que permite que los datos de audio de una trama “tomen prestado” espacio no utilizado de tramas anteriores mediante el mecanismo de **reservorio de bits** (sección 4.2)– así como los factores de escala y las tablas de Huffman seleccionadas para cada uno de los dos **gránulos** (*granules*) de 576 muestras que componen cada trama.

4.2 Arquitectura del Codificador

El códec MPEG-1 Layer III procesa el audio PCM de entrada en dos rutas paralelas, ilustradas en la Figura 3: una hacia el **banco de filtros**, que transforma la señal al dominio de la frecuencia, y otra hacia el **modelo psicoacústico**, que analiza esa misma señal (típicamente mediante una FFT independiente) para calcular umbrales de enmascaramiento.

Las etapas principales son:

Fig. 3: Diagrama de bloques del codificador MPEG-1 Layer III

- **Banco de filtros híbrido (polifase + MDCT):** primero, un banco de filtros polifase de 32 subbandas (512 coeficientes FIR) divide la señal en bandas de frecuencia de igual ancho, heredado de las Capas I y II. Sobre cada subbanda se aplica adicionalmente una **Transformada de Coseno Discreta Modificada** (MDCT) de 18 o 6 puntos (según el tipo de bloque), refinando la resolución en frecuencia hasta 576 líneas espectrales por gránulo. Esta combinación –exclusiva de Layer III– es la que le da su ventaja de compresión frente a las Capas I y II.
- **Conmutación de ventanas (*block switching*):** el codificador puede alternar entre bloques *largos* (mejor resolución en frecuencia, adecuados para señales estacionarias) y bloques *cortos* (mejor resolución temporal, usados en transitorios como percusiones, para evitar el artefacto de *pre-eco*), así como bloques *mixtos* que combinan ambos en distintas regiones espectrales de un mismo gránulo.
- **Codificación estéreo conjunta:** en modo *joint stereo*, el codificador puede aplicar **M/S stereo** (codificando la suma y diferencia de los canales L/R en lugar de L y R por separado) y/o **intensity stereo** (compartiendo información de envolvente espectral entre canales en las frecuencias altas, donde el oído es menos sensible a la localización precisa), reduciendo aún más la redundancia entre canales.
- **Cuantización no uniforme y doble lazo anidado:** los coeficientes MDCT se cuantizan con una ley de potencia (exponente $3/4$) que distribuye el error de cuantización de forma más perceptualmente uniforme que una cuantización lineal. Un **lazo de distorsión** interno ajusta los *factores de escala* por banda hasta que el ruido de cuantización cae por debajo del umbral de enmascaramiento, mientras que un **lazo de tasa** externo ajusta el tamaño del paso de cuantización global hasta que el número de bits resultante se ajusta al presupuesto disponible para el gránulo.
- **Codificación de Huffman:** los valores cuantizados se codifican mediante uno de 32 conjuntos de tablas de Huffman predefinidas por el estándar (codificación sin pérdida sobre datos ya cuantizados), seleccionando para cada gránulo las tablas que minimizan el número de bits resultante.
- **Reservorio de bits (*bit reservoir*):** dado que el flujo de salida debe respetar en promedio la tasa de bits nominal, pero algunos gránulos requieren más bits que otros, el codificador mantiene un “banco” de bits no utilizados en tramas simples que puede prestar a tramas más complejas, mejorando la calidad global sin abandonar una tasa de bits promedio constante.

4.3 Arquitectura del Decodificador

El decodificador realiza esencialmente el proceso inverso, y –a diferencia del codificador, cuyo diseño interno (modelo psicoacústico, estrategia de cuantización) queda abierto a cada implementación– está completamente especificado bit a bit por el estándar, garantizando que cualquier decodificador conforme produzca una salida equivalente:

1. Localización del *syncword* y validación de la cabecera de trama.
2. Lectura de la información lateral y, usando el puntero `main_data_begin`, recuperación de los datos principales (que pueden ubicarse físicamente en tramas anteriores gracias al reservorio de bits).

3. Decodificación de Huffman (proceso inverso al de codificación) para recuperar los valores cuantizados.
4. Descuantización (aplicando la ley de potencia inversa y los factores de escala) para recuperar una aproximación de los coeficientes espectrales.
5. Procesamiento estéreo inverso (deshacer *M/S* o *intensity stereo* según corresponda).
6. **IMDCT** (Transformada de Coseno Discreta Modificada Inversa) por subbanda, seguida de *traslape y suma* (*overlap-add*) para eliminar el aliasing introducido por la MDCT en el codificador.
7. **Banco de filtros de síntesis** (polifase inverso) que combina las 32 subbandas de vuelta en una señal PCM de tiempo continuo, lista para enviarse a un DAC.

Fig. 4: Diagrama de flujo del decodificador MPEG-1 Layer III

Esta asimetría –decodificador estrictamente especificado, codificador libre– es la razón por la cual codificadores de distintas épocas y calidades (como LAME) han podido mejorar sustancialmente su desempeño con el tiempo sin romper la compatibilidad con decodificadores existentes, y es también la que permite implementar decodificadores extremadamente ligeros en microcontroladores de recursos limitados.

5 Implementaciones

Para reproducir audio MP3 desde un microcontrolador existen, en la práctica, dos estrategias complementarias: delegar la decodificación a un **chip dedicado** vía un bus serial, o ejecutar un **decodificador de software** directamente sobre el núcleo del microcontrolador y enviar el PCM resultante a un DAC/códec externo mediante I2S.

5.1 Decodificación por Hardware Dedicado: VS1053b

El **VS1053b** (VLSI Solution) es un códec de audio de un solo chip que decodifica MP3, Ogg Vorbis, AAC, WMA y WAV/ADPCM íntegramente en su propio núcleo DSP interno (VS_DSP4), sin requerir ningún cómputo de decodificación por parte del microcontrolador anfitrión. Este último únicamente necesita transmitir el archivo MP3 en bruto (byte a byte) a través de una interfaz **SPI**, mientras el VS1053b se encarga de la decodificación completa y entrega el audio ya en una salida DAC estéreo integrada, lista para un amplificador de audífonos.

La interfaz del VS1053b se divide en dos canales SPI lógicos, seleccionados por líneas de *chip select* independientes:

- **SCI** (*Serial Control Interface*, pin XCS): para leer/escribir los registros de control de 16 bits del chip (modo de operación, volumen, tono de graves/agudos, etc.).
- **SDI** (*Serial Data Interface*, pin XDCS): para transmitir los datos comprimidos del archivo MP3.
- **DREQ**: línea de salida del VS1053b que indica al microcontrolador si el chip tiene espacio disponible en su buffer interno (al menos 32 bytes) para recibir más datos, evitando así un desbordamiento del buffer.

Fig. 5: Conexión SPI entre un microcontrolador STM32 y el módulo decodificador VS1053b

El siguiente ejemplo, basado en el driver HAL de STM32, ilustra la escritura de un registro SCI del VS1053b esperando explícitamente la señal DREQ antes de cada transacción, evitando saturar el buffer interno del chip:

Code: vs1053.c -- Escritura de registro SCI vía SPI/HAL

```
1  #define VS1053_SCI_WRITE_COMMAND  0x02
2
3  void VS1053_WriteRegister(uint8_t registerAddress, uint16_t
   registerValue)
4  {
5      uint8_t commandByte      = VS1053_SCI_WRITE_COMMAND;
6      uint8_t addressByte      = registerAddress;
7      uint8_t valueHighByte    = (uint8_t)(registerValue >> 8);
8      uint8_t valueLowByte     = (uint8_t)(registerValue & 0
   x00FF);
9
10     // Esperar a que el VS1053b este listo para recibir un
   comando SCI
11     while (HAL_GPIO_ReadPin(VS1053_DREQ_GPIO_Port,
   VS1053_DREQ_Pin) == GPIO_PIN_RESET)
12     {
13         // Espera activa; DREQ en alto indica buffer
   disponible
14     }
15
16     HAL_GPIO_WritePin(VS1053_XCS_GPIO_Port, VS1053_XCS_Pin,
   GPIO_PIN_RESET);
17
18     HAL_SPI_Transmit(&hspi2, &commandByte, 1, HAL_MAX_DELAY)
   ;
19     HAL_SPI_Transmit(&hspi2, &addressByte, 1, HAL_MAX_DELAY)
   ;
20     HAL_SPI_Transmit(&hspi2, &valueHighByte, 1, HAL_MAX_DELAY)
   ;
21     HAL_SPI_Transmit(&hspi2, &valueLowByte, 1, HAL_MAX_DELAY)
   ;
22
23     HAL_GPIO_WritePin(VS1053_XCS_GPIO_Port, VS1053_XCS_Pin,
   GPIO_PIN_SET);
24 }
25
26 void VS1053_SendAudioDataChunk(const uint8_t *mp3DataChunk,
   uint16_t chunkLength)
27 {
28     while (HAL_GPIO_ReadPin(VS1053_DREQ_GPIO_Port,
   VS1053_DREQ_Pin) == GPIO_PIN_RESET)
29     {
30         // Esperar espacio disponible en el buffer de datos
   del VS1053b
31     }
32
33     HAL_GPIO_WritePin(VS1053_XDCS_GPIO_Port, VS1053_XDCS_Pin,
   GPIO_PIN_RESET);
34     HAL_SPI_Transmit(&hspi2, (uint8_t *)mp3DataChunk,
   chunkLength, HAL_MAX_DELAY);
35     HAL_GPIO_WritePin(VS1053_XDCS_GPIO_Port, VS1053_XDCS_Pin,
   GPIO_PIN_SET);
36 }
```

Puntos clave del ejemplo:

- Cada transacción respeta explícitamente el protocolo de *handshake* por DREQ, evitando el uso de retardos fijos (*delays* arbitrarios) que podrían provocar pérdida de datos o bloqueos.
- El registro SCI_MODE (dirección 0x00) es el que típicamente se configura primero, estableciendo el bit SM_SDINew para usar el protocolo de datos moderno del chip.
- Al delegar toda la decodificación al VS1053b, el microcontrolador anfitrión queda libre para tareas adicionales (interfaz de usuario, lectura de tarjeta SD, control de otros periféricos), a costa de un mayor costo de materiales por incluir un chip dedicado.

5.2 Decodificación por Software: Decodificador Helix de Punto Fijo

Cuando no se desea (o no se puede) incluir un chip decodificador dedicado, es posible decodificar MP3 íntegramente en software sobre el propio núcleo Cortex-M, siempre que este cuente con una instrucción de multiplicación larga (32×32 bits a 64 bits) y suficiente memoria Flash/RAM. La opción más utilizada en proyectos embebidos es el **decodificador Helix MP3** de RealNetworks (posteriormente liberado como código abierto), disponible tanto en variante de **punto flotante** como de **punto fijo**; esta última está optimizada específicamente para procesadores ARM y no requiere unidad de punto flotante (FPU), lo que la hace viable incluso en microcontroladores de gama baja (p. ej. STM32F1). Existen también alternativas más compactas como **minimp3** (biblioteca de una sola cabecera, con soporte SIMD opcional) para casos donde se cuenta con más margen de cómputo.

El flujo típico de un reproductor MP3 por software en un STM32 combina tres subsistemas: lectura del archivo desde una tarjeta SD mediante **FatFs**, decodificación de tramas mediante Helix, y salida de las muestras PCM resultantes hacia un DAC/códec externo mediante el periférico **I2S** en modo DMA circular con doble buffer (*ping-pong*), de forma que mientras una mitad del buffer se transmite por DMA, la otra se rellena con la siguiente porción decodificada.

Fig. 6: Arquitectura de software de un reproductor MP3 embebido con decodificador Helix y salida I2S

Code: mp3_player.c -- Ciclo de decodificación con Helix y salida I2S por DMA

```

1 static HMP3Decoder    mp3DecoderHandle;
2 static MP3FrameInfo   mp3FrameInfo;
3
4 static uint8_t         compressedInputBuffer[
5     MP3_INPUT_BUFFER_SIZE];
6 static int16_t         pcmOutputBuffer[2][
7     PCM_HALF_BUFFER_SAMPLES];
8 static uint8_t         activePcmBufferHalf = 0;
9
10 void MP3Player_Init(void)
11 {
12     mp3DecoderHandle = MP3InitDecoder();
13 }
14 void MP3Player_DeCodeNextFrame(void)
15 {
16     uint8_t *readPointer = compressedInputBuffer;
17     int      bytesLeftInBuffer = MP3_INPUT_BUFFER_SIZE;

```

```

18 // Localizar el inicio de una trama valida dentro del
    buffer comprimido
19 int syncOffset = MP3FindSyncWord(readPointer,
    bytesLeftInBuffer);
20 if (syncOffset < 0)
21 {
22     FatFs_RefillCompressedBuffer(compressedInputBuffer,
        MP3_INPUT_BUFFER_SIZE);
23     return;
24 }
25
26 readPointer      += syncOffset;
27 bytesLeftInBuffer -= syncOffset;
28
29 int decodeResult = MP3Decode(mp3DecoderHandle,
30                             &readPointer,
31                             &bytesLeftInBuffer,
32                             pcmOutputBuffer[
                                activePcmBufferHalf],
33                             0);
34
35 if (decodeResult == ERR_MP3_NONE)
36 {
37     MP3GetLastFrameInfo(mp3DecoderHandle, &mp3FrameInfo);
38     activePcmBufferHalf ^= 1; // Alternar el buffer ping-
        pong
39 }
40
41 FatFs_ConsumeCompressedBytes(compressedInputBuffer,
    MP3_INPUT_BUFFER_SIZE - bytesLeftInBuffer);
42 }
43
44 // Callback disparado por HAL cuando el DMA de I2S termina de
    transmitir
45 // la primera o segunda mitad del buffer circular
46 void HAL_I2S_TxHalfCpltCallback(I2S_HandleTypeDef *hi2s)
47 {
48     MP3Player_DecodeNextFrame(); // Rellenar la mitad recién
        liberada
49 }
50
51 void HAL_I2S_TxCpltCallback(I2S_HandleTypeDef *hi2s)
52 {
53     MP3Player_DecodeNextFrame(); // Rellenar la otra mitad del
        buffer
54 }

```

Puntos clave del ejemplo:

- MP3FindSyncWord y MP3Decode corresponden a la API pública del decodificador Helix; el diseño explícito de sincronización de trama es necesario porque el flujo MP3 no garantiza alineación exacta entre los límites del buffer de lectura y los límites de trama.
- El uso de HAL_I2S_TxHalfCpltCallback y HAL_I2S_TxCpltCallback implementa el patrón de *doble buffer*: la decodificación de la siguiente porción de audio ocurre mientras el DMA transmite la porción ya decodificada, evitando cortes audibles (*glitches*) por bloqueos de CPU.
- Cada trama Layer III entrega siempre 1152 muestras por canal, por lo que el tamaño de pcmOutputBuffer debe dimensionarse en consecuencia (1152×2 canales para estéreo).

Table 2: Ventajas y desventajas del formato MP3

Ventajas	Desventajas
Compatibilidad prácticamente universal: décadas de soporte en hardware y software de todo tipo.	Eficiencia de compresión inferior a códecs modernos (AAC, Opus) a la misma tasa de bits.
Sin restricciones de patentes desde abril de 2017; libre para cualquier uso.	Formato exclusivamente perceptual/con pérdida; no apto para archivado de precisión o edición profesional intensiva.
Ecosistema maduro de decodificadores abiertos y ligeros (Helix, minimp3) aptos para microcontroladores de bajos recursos.	El decodificador está rígidamente especificado bit a bit; toda la innovación posible recae únicamente en el codificador.
Disponibilidad de silicio dedicado (VS1053b y similares) que descarga por completo el cómputo del microcontrolador anfitrión.	No soporta de forma nativa audio multicanal más allá de estéreo/dual mono (a diferencia de MPEG-2 Multichannel).
Estructura de trama autocontenida, ideal para streaming y reproducción desde cualquier punto del archivo.	Tasa de bits máxima limitada a 320 kbit/s, insuficiente para audio de muy alta resolución.
Codificadores de referencia libres y de alta calidad (LAME) ampliamente disponibles.	La codificación completa (modelo psicoacústico + doble lazo de cuantización) es computacionalmente costosa para ejecutarse en tiempo real en un microcontrolador; en la práctica casi siempre se opta por decodificar, no codificar, en el dispositivo embebido.

6 Ventajas y Desventajas

7 Aplicaciones y Uso Actual

Aunque su papel central en la distribución masiva de música ha sido parcialmente desplazado por el streaming con códecs más eficientes, MP3 mantiene un uso extendido en múltiples contextos:

- **Reproductores portátiles y dispositivos de consumo:** gran parte del catálogo musical distribuido comercialmente sigue disponible en MP3, y prácticamente todo reproductor de audio embebido conserva soporte para decodificarlo.
- **Sistemas embebidos de anuncios de voz:** paneles informativos, sistemas de transporte público y electrodomésticos que reproducen mensajes de voz pregrabados suelen almacenar dichos mensajes en MP3 por su balance entre tamaño de archivo y calidad de voz.
- **Juguetes y dispositivos IoT con audio:** módulos económicos basados en el VS1053b, DFPlayer Mini u otros decodificadores dedicados se emplean en proyectos de hobbistas y productos de bajo costo para reproducir efectos de sonido, alarmas o mensajes desde una tarjeta microSD.
- **Infoentretenimiento automotriz:** unidades principales (*head units*) que reproducen música desde memorias USB o tarjetas SD suelen incluir decodificación MP3 como formato mínimo garantizado.
- **Proyectos educativos y de branch IEEE:** el par VS1053b + STM32, o bien Helix + I2S, constituye una plataforma accesible para introducir a estudiantes de ingeniería electrónica en el procesamiento digital de señales de audio, el manejo de periféricos SPI/I2S y el uso de sistemas de archivos embebidos (FatFs) sobre tarjetas SD.

8 Referencias y Recursos para Profundizar

A continuación se listan fuentes primarias y recursos prácticos para profundizar en lo revisado en este reporte, organizados por categoría.

8.1 Estándares y especificaciones oficiales

- ISO/IEC, *ISO/IEC 11172-3:1993 – Information technology – Coding of moving pictures and associated audio for digital storage media at up to about 1.5 Mbit/s – Part 3: Audio* (vista previa oficial del estándar): https://webstore.ansi.org/preview-pages/ISO/preview_ISO+IEC+11172-3-1993.pdf
- Borrador de comité (CD) de ISO/IEC 11172-3, con la descripción detallada de la estructura de trama y las tres capas de codificación: https://course.ece.cmu.edu/~ece796/documents/MPEG-1_Audio_CD.doc
- ANSI Blog, *ISO/IEC 11172 Specifications for MPEG-1* (panorama de las partes del estándar, incluida la Parte 3 de audio): <https://blog.ansi.org/ansi/iso-iec-11172-mpeg-1-coding-15-mbits/>
- Fraunhofer IIS, página oficial del formato mp3 (historia y estado actual): <https://www.iis.fraunhofer.de/en/ff/amm/consumer-electronics/mp3.html>

8.2 Datasheets de hardware relevante

- VLSI Solution, *VS1053b – Ogg Vorbis/MP3/AAC/WMA/MIDI Audio Codec* (datasheet completo del decodificador dedicado): <https://www.vlsi.fi/fileadmin/datasheets/vs1053.pdf>
- Copia alternativa del datasheet VS1053b (Adafruit): <https://cdn-shop.adafruit.com/datasheets/vs1053.pdf>

8.3 Guías e implementación en STM32/HAL

- GitHub, *eziya/STM32_HAL_VS1053* – librería del VS1053 para el driver HAL de STM32: https://github.com/eziya/STM32_HAL_VS1053
- GitHub, *kattaliraees/VS1053-STM32* – ejemplo de interfaz de 7 líneas (SPI, DREQ, reset) entre VS1053/VS1063 y STM32: <https://github.com/kattaliraees/VS1053-STM32>
- Phipps Electronics, *How to Play a WAV File on STM32* – referencia práctica de reproducción de audio PCM vía I2S/DMA con FatFs sobre SD, extensible al buffer de salida de un decodificador MP3: <https://www.phippselectronics.com/how-to-play-a-wav-file-on-stm32/>
- Stm32World Wiki, *STM32 Music Player* – notas de diseño sobre configuración de frecuencia I2S y lectura de encabezados de audio: https://stm32world.com/wiki/STM32_Music_Player
- ControllersTech, *STM32 WAV Player Using USB Host* – ejemplo completo de I2S + FatFs + códec CS43L22 en placas STM32 Discovery: <https://controllerstech.com/waveplayer-using-stm32-discovery/>

8.4 Librerías y herramientas de software

- GitHub, *chmorgan/libhelix-mp3* – decodificador MP3 de punto fijo de RealNetworks, optimizado para ARM: <https://github.com/chmorgan/libhelix-mp3>

- Silicon Labs, *AN1112: Helix MP3 Decoder on Series 1 EFM32 MCU Devices* – nota de aplicación con la integración completa del decodificador Helix sobre un microcontrolador de bajos recursos: <https://www.silabs.com/documents/public/application-notes/an1112-efm32-helix-mp3-decoder.pdf>
- GitHub, *pschatzmann/arduino-libhelix* – envoltorio de Helix orientado a proyectos Arduino, ESP32 y STM32 con salida I2S: <https://github.com/pschatzmann/arduino-libhelix>
- GitHub, *lieff/minimp3* – decodificador MP3 minimalista de una sola cabecera, alternativa ligera a Helix: <https://github.com/lieff/minimp3>
- SourceForge, *LAME MP3 Encoder* – codificador MP3 de referencia, libre de patentes desde 2017: <https://lame.sourceforge.io/>

9 Conclusión

MP3 se mantiene, más de tres décadas después de su concepción, como uno de los formatos de audio comprimido más ubicuos gracias a la combinación de un diseño técnico sólido –banco de filtros híbrido, modelo psicoacústico y codificación entrópica– con un decodificador rigurosamente especificado que ha permitido implementaciones extremadamente ligeras y confiables. Para el trabajo con microcontroladores, esta madurez se traduce en dos caminos igualmente viables: delegar la decodificación a silicio dedicado como el VS1053b, minimizando la carga de cómputo del sistema anfitrión, o ejecutar un decodificador de software de punto fijo como Helix directamente sobre el núcleo Cortex-M, apoyado en una salida I2S con DMA. La expiración de todas las patentes asociadas en 2017 elimina además cualquier barrera de licenciamiento, consolidando a MP3 como una opción práctica y de bajo riesgo legal para proyectos de instrumentación, dispositivos IoT con audio, y trabajos educativos en sistemas embebidos.